

Sistemas de Numeracao e Representacao de Dados

Resumo de estudo — Bits, armazenamento, bases numericas, conversoes e aritmetica binaria

■ **Objetivo do resumo** — Compreender a representacao de dados em nivel de hardware (bits e bytes), entender por que o computador trabalha em binario, dominar os sistemas de numeracao posicional (binario, octal, hexadecimal, BCD), as tecnicas de conversao entre bases, a representacao de caracteres, a aritmetica binaria e a forma sinal-magnitude.

Secao 1 — Conceitos Basicos: Bits

■ **Bit (Binary Digit)** — A menor unidade de informacao em um computador. Assume apenas dois valores: **0** ou **1**. Toda informacao digital — texto, imagem, som, video — e, em ultima instancia, uma sequencia de bits.

1. Por que o Computador Nao Trabalha em Decimal?

Os circuitos eletronicos do computador operam com dois estados eletricos (ligado/desligado, alta/baixa tensao). Representar 10 estados distintos (0 a 9) exigiria circuitos muito mais complexos e sujeitos a erros. O sistema binario (2 estados) e naturalmente compativel com a eletronica digital.

2. Armazenamento e Unidades de Memoria

■ **Memoria** — Organizada como um total de 2^n celulas enderecaveis. Cada celula armazena tipicamente 1 byte (8 bits).

Unidade	Símbolo	Equivalencia	Potencia de 2
Byte	B	8 bits	2^0 bytes = 1 B
Kilobyte	KB	1.024 bytes	2^{10} bytes
Megabyte	MB	1.024 KB	2^{20} bytes
Gigabyte	GB	1.024 MB	2^{30} bytes
Terabyte	TB	1.024 GB	2^{40} bytes

■ **Atencao** — Na computacao, $1 \text{ KB} = 1.024 \text{ bytes}$ (2^{10}), nao 1.000. Isso porque as memorias sao dimensionadas em potencias de 2.

Secao 2 — Sistemas de Numeracao Posicional

1. Conceitos Fundamentais

Numero — conceito abstrato de quantidade. **Numeral** — representacao escrita de um numero. **Algarismo** — cada simbolo individual usado no numeral.

■ **Sistema posicional** — O valor de cada algarismo depende de sua **posicao**. Em uma base b , um numeral representa:

$$N = a_{n-1} \cdot b^{n-1} + \dots + a_1 \cdot b^1 + a_0 \cdot b^0 + a_{-1} \cdot b^{-1} + \dots$$

• Exemplo (base 10): $2105 = 2 \cdot 10^3 + 1 \cdot 10^2 + 0 \cdot 10^1 + 5 \cdot 10^0$

2. Visao Geral das Bases

Base	Nome	Algarismos	Bits/digito	Uso principal
2	Binario	0, 1	1	Representacao interna do computador
8	Octal	0 a 7	3	Permissoes Unix; notacao compacta
10	Decimal	0 a 9	~3,32	Sistema humano do dia a dia
16	Hexadecimal	0-9, A-F	4	Enderecos de memoria; cores (HTML/CSS)

Secao 3 — Sistema Binario (Base 2)

1. Conversao Binario → Decimal

Multiplique cada digito pela potencia de 2 correspondente a sua posicao e some tudo.

■ Exemplo (inteiro):

$$(1101001)_2 = 1 \cdot 2^0 + 0 \cdot 2^1 + 0 \cdot 2^2 + 1 \cdot 2^3 + 0 \cdot 2^4 + 1 \cdot 2^5 + 1 \cdot 2^6 = 1 + 8 + 32 + 64 = 105$$

■ Exemplo (fracionario):

$$(110,1001)_2 = 1 \cdot 2^{-4} + 1 \cdot 2^{-1} + 1 \cdot 2^1 + 1 \cdot 2^2 = 1/16 + 0,5 + 2 + 4 = 6,5625$$

2. Conversao Decimal → Binario (Divisoes Sucessivas)

■ **Metodo** — Divida o numero inteiro por 2 sucessivamente, anotando os restos. A leitura dos restos **de tras para frente** forma o numeral binario.

■ Exemplo: $(731)_{10} \rightarrow (?)_2$

n	731	365	182	91	45	22	11	5	2	1	0
q=n/2	365	182	91	45	22	11	5	2	1	0	-
resto	1	1	0	1	1	0	1	1	0	1	-

Lendo de tras para frente: $(731)_{10} = (1011011011)_2$

■ Exemplo: $(513)_{10} \rightarrow (?)_2$

n	513	256	128	64	32	16	8	4	2	1	0
q=n/2	256	128	64	32	16	8	4	2	1	0	-
resto	1	0	0	0	0	0	0	0	0	1	-

Lendo de tras para frente: $(513)_{10} = (1000000001)_2$

Secao 4 — Sistema Octal (Base 8)

■ **Octal** — Usa algarismos de 0 a 7. Cada dígito octal equivale a exatamente **3 bits** (pois $2^3 = 8$). Potências: $8^0=1$, $8^1=8$, $8^2=64$, $8^3=512$...

1. Conversão Octal → Decimal

■ **Exemplo:**

$$(135)_8 = 1 \cdot 8^2 + 3 \cdot 8^1 + 5 \cdot 8^0 = 64 + 24 + 5 = 93_{10}$$

2. Conversão Decimal → Octal

■ **Exemplo:** $(157)_{10} \rightarrow (?)_8$

$157 / 8 = 19$ (resto **5**) $\rightarrow 19 / 8 = 2$ (resto **3**) $\rightarrow 2 / 8 = 0$ (resto **2**)

Lendo de trás para frente: $(157)_{10} = (235)_8$

Seção 5 — Sistema Hexadecimal (Base 16)

■ **Hexadecimal** — Usa algarismos de 0 a 9 e letras A a F (A=10, B=11, C=12, D=13, E=14, F=15). Cada dígito hex equivale a **4 bits** (pois $2^4 = 16$).

1. Tabela Hex ↔ Decimal ↔ Binário

Hex	Dec	Bin		Hex	Dec	Bin
0	0	0000		8	8	1000
1	1	0001		9	9	1001
2	2	0010		A	10	1010
3	3	0011		B	11	1011
4	4	0100		C	12	1100
5	5	0101		D	13	1101
6	6	0110		E	14	1110
7	7	0111		F	15	1111

2. Conversão Hex → Decimal

$$(2A6)_{16} = 2 \cdot 16^2 + 10 \cdot 16^1 + 6 \cdot 16^0 = 512 + 160 + 6 = 678_{10}$$

3. Conversão Decimal → Hex

■ **Exemplo:** $(250)_{10} \rightarrow (?)_{16}$

$250 / 16 = 15$ (resto **10 = A**) $\rightarrow 15 / 16 = 0$ (resto **15 = F**)

Lendo de trás para frente: $(250)_{10} = (FA)_{16}$

Seção 6 — Código BCD (Binary-Coded Decimal)

■ **BCD** — Cada **dígito decimal** de um número é substituído individualmente pelo seu equivalente binário de **4 bits**. Diferente da conversão direta para binário!

■ **Exemplo:** $(459)_{10}$ em BCD

Digito decimal	4	5	9
BCD (4 bits)	0100	0101	1001

Resultado: $(459)_{10} = 0100\ 0101\ 1001$ (BCD)

■ **BCD vs. Binário puro** — No binário puro, $(459)_{10} = (111001011)_2$ (9 bits). Em BCD, são 12 bits (4 por dígito x 3 dígitos). O BCD é menos eficiente em armazenamento, mas facilita a conversão direta para displays decimais (calculadoras, relógios digitais).

Seção 7 — Conversão de Números Fracionários

■ **Princípio** — Um número fracionário $N = N_i + N_f$ (parte inteira + parte fracionária) é convertido em duas etapas: parte inteira por **divisões sucessivas** e parte fracionária por **multiplicações sucessivas** pela base de destino.

1. Método das Multiplicações Sucessivas

Multiplique repetidamente por b , anotando a parte inteira de cada resultado. A leitura é feita **de cima para baixo**.

■ **Exemplo:** $(12,463)_{10} \rightarrow (?)_2$

Parte inteira: $12_{10} = (1100)_2$ (divisões por 2)

Parte fracionária (multiplicações por 2):

Multiplicação	Resultado	Dígito
$0,463 \times 2$	0,926	0
$0,926 \times 2$	1,852	1
$0,852 \times 2$	1,704	1
$0,704 \times 2$	1,408	1

Resultado: $(12,463)_{10} \approx (1100,0111)_2$

2. Exemplo Completo — $(246,12)_{10}$ em Três Bases

■ **Para binário:**

Inteira: $246_{10} = (11110110)_2$. Fracionária: $0,12 \approx (0,00011)_2$.

Resultado: $(246,12)_{10} \approx (11110110,00011)_2$

■ **Para octal (multiplicações por 8):**

Multiplicação	Resultado	Dígito
$0,12 \times 8$	0,96	0
$0,96 \times 8$	7,68	7
$0,68 \times 8$	5,44	5
$0,44 \times 8$	3,52	3

Inteira: $246_{10} = (366)_8$. **Resultado:** $(246,12)_{10} \approx (366,0753)_8$

■ **Para hexadecimal (multiplicações por 16):**

Multiplicacao	Resultado	Digito
0,12 x 16	1,92	1
0,92 x 16	14,72	E
0,72 x 16	11,52	B

Inteira: $246_{10} = (F6)_{16}$. **Resultado:** $(246,12)_{10} \approx (F6,1EB)_{16}$

Secao 8 — Conversoes Diretas (Casos Especiais)

■ **Atalho fundamental** — Como $8 = 2^3$ e $16 = 2^4$, a conversao entre binario e octal/hex e feita por simples **agrupamento de bits**, sem precisar passar pelo decimal.

1. Binario ↔ Octal (grupos de 3 bits)

■ **Exemplo:**

Binario	111	010	011
Octal	7	2	3

Logo: $(111010011)_2 = (723)_8$

2. Binario ↔ Hexadecimal (grupos de 4 bits)

■ **Exemplo:**

Binario	0011	0110	1010
Hex	3	6	A

Logo: $(001101101010)_2 = (36A)_{16}$

■ **Cuidado com o preenchimento!** — Antes da virgula: complete com zeros **a esquerda**. Depois da virgula: complete com zeros **a direita**.

Secao 9 — Representacao de Caracteres

Computadores tambem representam letras e simbolos usando bits, seguindo tabelas padronizadas.

Padrao	Bits	Capacidade	Uso
ASCII	7 bits	128 caracteres (letras, digitos, simbolos)	Padrao classico. A = 65, a = 97
Unicode	Variavel (8, 16 ou 32 bits)	Mais de 140 mil caracteres	Base dos sistemas modernos. Suporta quase todos os idiomas.
EBCDIC	8 bits	256 caracteres	Sistema antigo, usado em mainframes IBM.

Secao 10 — Aritmetica Binaria

1. Soma Binaria

As regras da adicao binaria sao:

Operacao	Resultado	Observacao
0 + 0	0	-
0 + 1	1	-
1 + 0	1	-
1 + 1	10	0, vai 1 (carry)
1 + 1 + 1	11	1, vai 1 (carry)

■ **Exemplo:**

$$(101011)_2 + (100111)_2 = (1010010)_2$$

2. Complemento de 2 — Representacao de Negativos

■ **Complemento de 2** — Metodo usado pelos computadores reais para representar numeros negativos em binario. Permite realizar subtracao usando apenas o circuito de soma.

Como obter o complemento de 2:

■ **Passo 1:** Inverter todos os bits (complemento de 1).

■ **Passo 2:** Somar 1 ao resultado.

$$\text{Comp2}(N) = \text{NOT}(N) + 1$$

■ **Exemplo: representar -3 em 4 bits**

Etapa	Valor	Resultado
3 em binario (4 bits)	-	0011
Inverter bits (comp. 1)	0011 →	1100
Somar 1 (comp. 2)	1100 + 1 =	1101

Logo: $(-3)_{10} = (1101)_2$ em complemento de 2.

3. Subtracao Usando Complemento de 2

■ **Principio** — Em vez de calcular $A - B$, o computador calcula $A + \text{Comp2}(B)$. O carry excedente (bit extra a esquerda) e descartado.

■ **Exemplo:** $(110001)_2 - (010011)_2$

Etapa	Operacao	Resultado
1. Igualar bits	Ambos com 6 bits	110001 e 010011
2. Comp. de 1 de B	Inverter 010011	101100
3. Comp. de 2 de B	101100 + 1	101101
4. Somar A + Comp2(B)	110001 + 101101	1 011110
5. Descartar carry	Remover o 1 a esquerda	011110

Resultado: $(110001)_2 - (010011)_2 = (011110)_2$

Secao 11 — Numeros Sinalizados: Forma Sinal-Magnitude

■ **Forma Sinal-Magnitude** — Reserva um bit da palavra binária exclusivamente para representar o **sinal** do número. Os bits restantes representam a **magnitude** (valor absoluto).

1. Estrutura

Em uma palavra de n bits:

■ **Bit de sinal** — O bit mais significativo (mais a esquerda): **0** = **positivo (+)**, **1** = **negativo (-)**.

■ **Bits de magnitude** — Os $n - 1$ bits restantes representam o valor absoluto do número.

Bit de sinal	Bits de magnitude ($n - 1$ bits)
0 ou 1	Valor absoluto do número em binário

2. Exemplo Resolvido

Determine o valor decimal do número binário em sinal-magnitude: 10010101

Etapa	Operação	Resultado
1. Identificar sinal	Bit mais a esquerda = 1	Negativo (-)
2. Extrair magnitude	Restante: 0010101	0010101
3. Converter magnitude	$0 \cdot 2^4 + 1 \cdot 2^2 + 1 \cdot 2^0$	$16 + 4 + 1 = 21$
4. Aplicar sinal	- (21)	-21

Resultado: $(10010101)_{SM} = -21_{10}$

3. Sinal-Magnitude vs. Complemento de 2

Aspecto	Sinal-Magnitude	Complemento de 2
Bit de sinal	Sim (MSB)	Sim (MSB), mas integrado ao cálculo
Faixa (n bits)	$-(2^{n-1} - 1)$ a $+(2^{n-1} - 1)$	$-(2^{n-1})$ a $+(2^{n-1} - 1)$
Zero duplo?	Sim: +0 (0000) e -0 (1000)	Não: apenas 0000
Aritmética	Complexa: precisa tratar sinal separadamente	Simples: soma direta funciona para + e -
Uso real	Pouco usado em CPUs modernas	Padrão nas CPUs atuais

■ **Por que Complemento de 2 venceu?** — Sinal-magnitude tem dois zeros (+0 e -0), o que complica comparações. Além disso, a soma/subtração exige circuitos extras para verificar sinais. Em complemento de 2, a mesma ULA que soma números positivos já funciona para negativos — sem circuitos adicionais.

Síntese — Mapa do Resumo

Secao	Tema	Conceito-chave
1	Bits e Armazenamento	Bit = 0 ou 1; KB = 1024 bytes
2	Numeracao Posicional	Valor = posicao x potencia da base
3	Sistema Binario	Divisoes por 2; leitura dos restos
4	Sistema Octal	1 digito octal = 3 bits
5	Sistema Hexadecimal	1 digito hex = 4 bits; A=10...F=15
6	Codigo BCD	Cada digito decimal → 4 bits
7	Fracoes entre Bases	Inteira: divisoes; fracionaria: multiplicacoes sucessivas
8	Conversoes Diretas	Bin↔Oct: 3 bits; Bin↔Hex: 4 bits
9	Repr. de Caracteres	ASCII (7 bits), Unicode, EBCDIC
10	Aritmetica Binaria	Soma binaria; complemento de 2
11	Sinal-Magnitude	MSB = sinal (0=+, 1=-); resto = mag.

Resumo Rapido — Cola de Estudo

■ **Checklist** — • Explicar por que o computador usa base 2 (dois estados eletricos). • Converter KB, MB, GB usando potencias de 2. • Definir sistema posicional e expandir um numeral em qualquer base. • Converter binario → decimal (somar potencias de 2). • Converter decimal → binario (divisoes sucessivas por 2). • Converter a parte fracionaria pelo metodo das multiplicacoes sucessivas. • Converter octal ↔ decimal e hexadecimal ↔ decimal. • Usar atalhos: binario ↔ octal (3 bits) e binario ↔ hex (4 bits). • Saber a tabela hex (A=10 ... F=15) de memoria. • Codificar um numero decimal em BCD (4 bits por digito). • Distinguir ASCII, Unicode e EBCDIC. • Realizar soma binaria com carry. • Obter o complemento de 2: inverter bits + somar 1. • Realizar subtracao binaria via $A + \text{Comp2}(B)$. • Interpretar um numero em forma sinal-magnitude (MSB = sinal). • Comparar sinal-magnitude vs. complemento de 2.

■ **Regras de ouro** — 1) Parte inteira → divisoes sucessivas (restos de tras para frente). Parte fracionaria → multiplicacoes sucessivas (inteiros de cima para baixo). 2) Octal ↔ binario: grupos de 3 bits. Hex ↔ binario: grupos de 4 bits. 3) BCD ≠ binario puro: $(459)_{10}$ em BCD = 0100 0101 1001, mas em binario = 111001011. 4) $\text{Comp2}(N) = \text{NOT}(N) + 1$ — transforma subtracao em soma. 5) Na subtracao por complemento de 2, o carry excedente e sempre descartado. 6) Sinal-magnitude tem dois zeros (+0 e -0) — complemento de 2 tem apenas um.

■ **Conexao com outras areas** — Permissoes Unix usam octal (chmod 755). Enderecos de memoria e cores HTML/CSS usam hexadecimal. Unicode e essencial para textos em qualquer idioma. Complemento de 2 e o padrao real das CPUs para aritmetica com sinal — entender esse mecanismo e fundamental para programacao de baixo nivel e depuracao.

*#arquitetura #bits #binario #octal #hexadecimal #bcd #ascii #unicode #complemento2
#sinalmagnitude*